

Using the Agile Unified Process in Banking

Ioannis T. Christou, *Athens Information Technology*

Stavros T. Ponis, *National Technical University of Athens*

Eleni Palaiologou, *Athens Information Technology*

A banking-sector project combined agile methods with the Rational Unified Process to exploit service-oriented architecture functionality and user interface integration of legacy applications.

The banking sector is well known for using large, sometimes monolithic, legacy systems. Now, banks find themselves having to catch up with rapid advancements in software development that call for new service-oriented computing paradigms. Unfortunately, this task is nontrivial and often requires huge projects that are costly, time consuming, and risky. The safe choice for a development methodology is a process framework such as the Rational Unified Process (RUP), which is customizable enough to fit any project. However, customizing RUP isn't trivial,

so it's often used in its full-blown out-of-the-box form, which entails significant sacrifices of time, cost, and flexibility.^{1–3} But what if the organization is unwilling to make these sacrifices? What happens when time, cost, and flexibility aren't negotiable issues but hard management constraints?

We recently applied the Agile Unified Process (AUP)⁴—a hybrid approach designed by Scott Ambler combining RUP with agile methods^{5–7}—to a successful project in the banking sector. The project achieved on-time delivery within budget, integrating heavy legacy back-end application systems with newly reengineered client user-interface applications on a modern service-oriented architecture (SOA) platform.⁸

The Agile Unified Process

Agile methods and RUP are among the most widely used software development methods. Although both are iterative, they seem to have significant differences.

For starters, RUP is a process framework offering adaptability to every software development

project, regardless of its size. Still, customizing RUP demands both experienced staff and preparation time, otherwise it could result in an oversimplified project or, worse, a failure to meet essential requirements. To avoid the effort and risk, organizations often implement RUP with little or no customization, resulting in a complex process requiring numerous documentation artifacts. On the other hand, agile methods are considered more lightweight, emphasizing people and shifting authority to the developers at the heart of the project. Nevertheless, the two methods aren't incompatible.

AUP is an agile public-domain instantiation of RUP.^{9–11} It's a simple, easy-to-understand approach to developing business-related software using agile techniques and concepts while remaining true to RUP. It applies agile techniques including agile modeling (AM), test-driven design (TDD), agile change management, and database refactoring to improve productivity. AM is the basis for agile model-driven development and is a key concept introduced in AUP to effectively model and document software systems. Simply put, AM is a collection of

values, principles, and practices for modeling software in a lightweight manner.⁹⁻¹⁰ It values communication, simplicity, courage, feedback, and humility. Its principles include

- *assuming simplicity*, which is the equivalent of Occam's razor in software engineering;
- *embracing change*, in that requirements will inevitably change during the project lifecycle;
- *incremental change* of a system over time to accommodate requirement changes; and
- *rapid feedback* from project stakeholders after every software release.

AM practices are a set of best practices that include creating several models in parallel, modeling in small increments, iterating to another artifact, and updating models only when it hurts.

As in RUP, AUP has four major phases:

1. *Inception*. The team identifies the project's initial scope, a potential architecture, and obtains initial funding and stakeholder acceptance.
2. *Elaboration*. The team establishes the system's feasibility and proposed architecture.
3. *Construction*. The team builds working software on a regular, incremental basis that meets the project stakeholders' highest-priority needs.
4. *Transition*. The team validates and deploys the system in the production environment.

Unlike RUP however, AUP is guided by seven principles:

- *Understanding*. Understand the business of the organization and the problem domain the project addresses, and identify a viable solution.
- *Implementation*. Transform models into executable code, and perform a basic level of testing, particularly unit testing.
- *Testing*. Find defects, validate that the system works as designed, and verify that it meets the requirements.
- *Deployment*. Plan for the system's delivery, and execute the plan to make it available to end users.
- *Configuration management*. Manage access to project artifacts. This includes tracking artifact versions over time and then controlling and managing changes to them.
- *Project management*. Direct the activities that take place within the project. This includes managing risks, directing people (assigning tasks, tracking progress, and so on), and coordinating with people and systems outside the

project's scope to ensure that it's delivered on time and within budget.

- *Environment*. Support the rest of the effort by ensuring that the team has the proper process, guidance (standards and guidelines), and tools (hardware, software, and so on).

Ambler designed AUP around a few basic concepts. First, people won't read detailed process documentation, but they will want high-level guidance and training from time to time. Second, describe things concisely in a few pages, not thousands. Third, focus on the important activities and risks, not every minute detail. Fourth, teams should use any tools they prefer for a given job, not have tools imposed on them by the process. And finally, the process should be highly customizable so that you can tailor it to a specific project's needs.

The Integrated Desktop

Using AUP, one of the largest banks in Greece implemented a small to medium-scale project called the Integrated Desktop (ID). It was an important part of a large company-wide project that aimed to transition the company's IT architecture from the aged client-server model to one using SOA concepts. The parent project was based on an enterprise service bus framework—a standard architectural framework in the world of SOAs—developed for the company's particular needs.

The project's main objective was to host private-banking applications with single sign-on capability, thus automating daily tasks via global customer handling, exploiting the enterprise service bus framework architecture, and managing multiple and concurrent customer sessions. Modifying existing functionality or creating new functionality in the existing back-end systems was beyond the project's scope.

In the context of the parent project, the bank adopted RUP as its development methodology. ID was the first to adopt AUP to produce quick-win user results. The bank's IT strategy unit and IT development units authorized AUP's adoption.

Figure 1 summarizes the project's main characteristics.

Project Life Cycle Based on AUP

The team followed the SOA-phased deployment approach based on AUP and Microsoft's Smart Client and Customer Care Framework (CCF).⁹ The ID phased deployment stopped at CCF step 2. The project included an integrated user interface that combined existing user interfaces and new applications based on a small set of enterprise service



Project name	Integrated Desktop
Project owner	Private banking unit
Bank units involved in the project	IT development and operations, organization and planning, private banking (top management, relationship managers, middle office, customer documentation, and MIS)
Team size	Nine developers, three technicians (operational and administrative support), two test managers, five users, and four top managers (see steering committee)
Steering committee	Project manager, IT development manager, and top management review (private-banking operations unit manager, brands unit manager, area manager, and private-banking general manager)
Project manager	An author of this article who has much experience in successfully managing large-scale IT projects
Project duration	Six months
Project size	13 use cases

Figure 1. Project information summary. The project involved 23 people spanning a large portion of the organizational hierarchy. In addition to the purely technical core, the people involved included five users and four top managers who formed the project steering committee. The project lasted six months, during which the team implemented a total of 13 use cases.

The team followed AUP's four phases throughout the project life cycle. Figure 3 shows the project time line.

Inception

During inception, the team defined the project scope as follows:

- A role-based integrated user desktop hosts private-banking applications.
- The ID system automates daily tasks via global customer handling, exploiting the SOA-based parent project architecture and managing multiple and concurrent customer sessions.
- The ID system prefetches customer information and shares it intelligently among different applications to eliminate redundancy and radically reduce the time users must spend to complete tasks.

Using AUP, the team identified more than 10 use cases and key requirements in this phase. The project manager made cost and schedule estimates used for iterations in later phases (mostly during elaboration). This doesn't mean that she planned out the entire project at this point. As with all planning, those tasks scheduled for completion in the near future were detailed more accurately and with greater confidence, whereas tasks further down the line were understood to be estimates with larger margins of error.

The team also defined the project's risks during inception. The list of risks was a living compilation that changed over time. Risks drive project management, just as the highest-priority risks drive iteration scheduling. Earlier iterations addressed higher-priority risks. Difficulties the team initially identified included

- communicating the new architecture's benefits to the users and producing really useful results;
- meeting scheduling requirements due to the project's innovative approach;
- maintaining management and user commitment throughout all phases; and

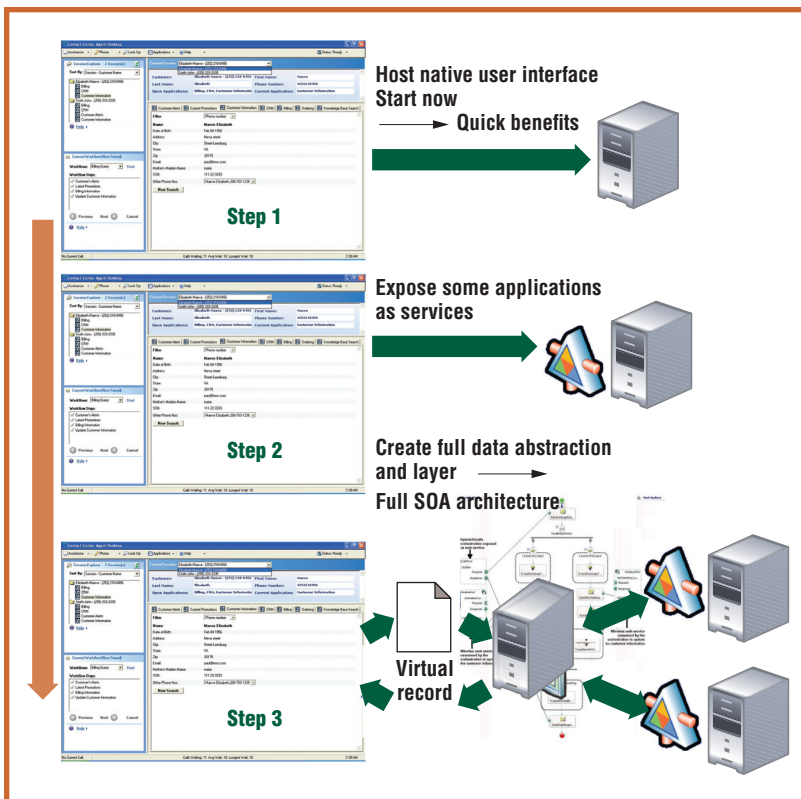


Figure 2. Development steps of the Integrated Desktop (ID) project. ID was based on the Microsoft Customer Care Framework offering Single Sign-On functionality for all applications it hosted using the Citrix Password Manager.

bus services. The idea was to deliver a pilot project and proceed with a new multiphased project based on the user response and acceptance (see Figure 2).

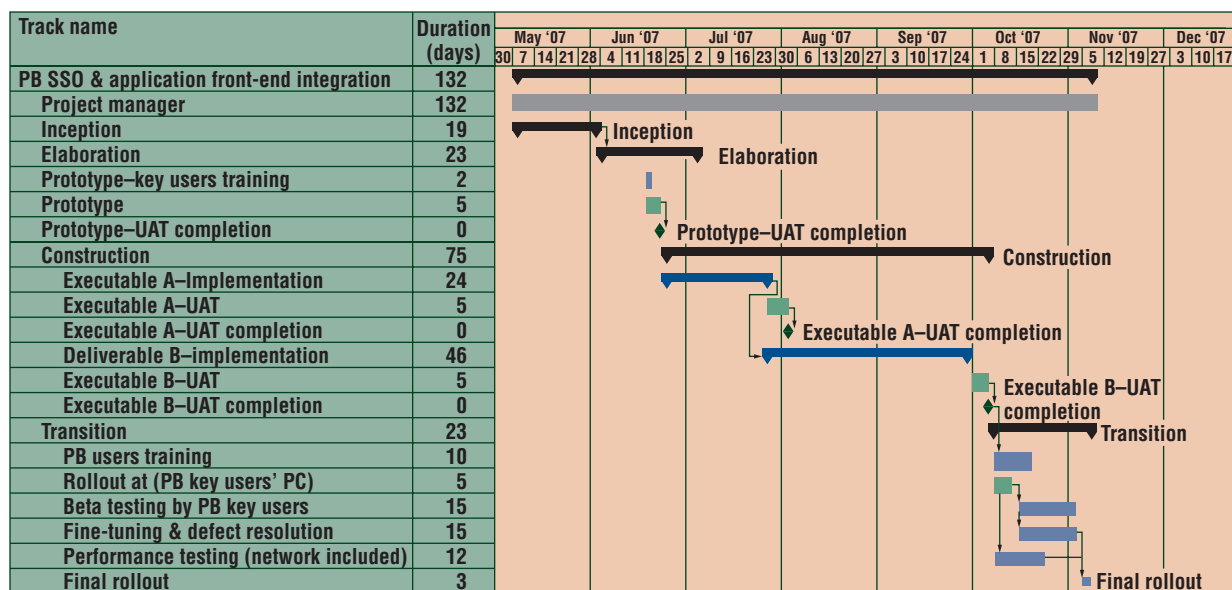


Figure 3. Project time line. The inception phase took 19 days, the elaboration phase lasted 23 days, the construction phase took 75 days, and the transition phase lasted another 23 days. PB stands for private banking. SSO stands for single sign-on. UAT stands for user acceptance test.

■ potential incompatibility between project requirements and the bank's culture and infrastructure.

Finally, the project needed to make sense from a technical, operational, and business perspective. The project manager collaborated with the development, architectural, and operations teams to create time and cost estimates and the steering committee approved the proposal. In general, internal banking project management allows a 20 percent margin for time and cost estimates. Projects that stay within this limit are deemed successful.

Elaboration

This phase aims to prove the system's architecture by elaborating a well-accepted working skeleton called an architectural prototype. The point was to ensure that the team could actually develop a system that satisfied the requirements. The team's business analyst gathered requirements. (Still, the users themselves prepared the current system's "as-is" documentation, proving in some sense their commitment to the project.) Then, the development team prepared a mock-up demo with screenshots for each software function in a sequence following the business workflow and presented the ID user interface. The team then held user workshops—one with each user representative per unit and one global workshop with all units, gathering their comments, remarks, and suggestions on the user interface and business workflow and system functionality. The project manager sent debriefings of the workshops' re-

sults to the users for confirmation, which in turn produced the necessary results for iteration 2. The team made the changes and top management signed off. The team also held a user-acceptance-testing workshop during which they received a second round of minor correction requests.

The requirements weren't completely specified at this point. They were detailed only so as to ensure an understanding of the architectural risks of each requirement's scope for subsequent planning. The team identified and prioritized the risks, addressing significant ones during elaboration. Addressing architectural risks can take several forms: research into similar systems, a stand-alone test suite, a working prototype, and so on.

In our case, a working prototype showcasing the architecture and user interface functionality was developed that enabled project management to mitigate architectural and usability risks. The ID hosted core banking, portfolio management, customer relationship management, fraud detection, suspicious activity reporting, document management, and a management information system for private banking. The team captured most of the system requirements. Common processes undertaken in this phase included creating process-flow, use-case, and sequence diagrams.

The team wrote use cases during the elaboration phase; each one would become the start of a new construction iteration. The user members of the team described in detail all the daily tasks performed until then, and the technical members of the team documented them. The document

**The team
needed to
start thinking
outside of the
current UML-
centric nature
of RUP.**

describing the “as-is” situation was a main deliverable. After detailed study and architectural design finalization, the project manager produced a similar document with the “to-be” situation. She extensively conferred with the users in developing the final version of this document.

Construction

This phase develops the system so that it’s ready for preproduction testing. Previous phases identified most requirements and created a baseline for the system’s architecture. Emphasis then shifted to prioritizing and understanding the requirements, brainstorming a solution, and coding and testing the software. Construction was the project’s largest phase. In this phase, the team built the remainder of the system on the foundation laid during elaboration. The team implemented system features in two time-boxed iterations lasting 24 and 46 working days.

Each iteration resulted in an executable release of the software. The team deployed an early release of the system to obtain user feedback. The team implemented 30 percent of the process flow automations (the most complicated ones) during the first iteration and included them in executable A. They included corrections to Executable A and all remaining process flow automation in executable B. The successful completion of each of the two iterations (not including automated unit and integration testing) ended with a user acceptance test (UAT). This ensured that progress was always based on mutual agreement with the users.

To exit the construction phase, the project had to pass the initial operational-capability milestone. The primary issue was whether the current version was ready to move into the preproduction test environment for system and acceptance testing. The team achieved that milestone with a presentation demo during a workshop with the members of the steering committee.

Transition

This phase lasted 23 days. Transition includes system and acceptance testing and focuses on delivering the system into production. In this phase, the team gradually deployed the system to the target users. Feedback from an initial release resulted in further refinements that they incorporated over the course of two transition phase iterations.

Extensive testing occurred, including beta testing. Fine-tuning of the existing hosted applications and integrated desktop took place as well as rework to address significant defects. Al-

though users and test managers tested integration early enough, during the elaboration (hosting of applications) and construction (UAT in the testing environment) phases, differences in set-up between preproduction or production and UAT environment couldn’t be predicted. Most of the issues resulted from special settings of the hosted applications in the preproduction or production environment. The time and effort spent in the transition phase was significant owing to the fine-tuning required for the entire infrastructure to improve performance and assure smooth, stable operation.

The phased-rollout approach that came next included extensive beta testing by small groups before the company released the system to the whole end-user population, including the final technical documentation with detailed physical-architecture and technical-user manuals. The company conducted exhaustive training programs, addressing both operation and support.

To exit the transition phase, the project team had to pass the product release milestone, which included the following items. Business stakeholder acceptance: the business stakeholders were satisfied with and accepted the system. Operations acceptance: the people responsible for operating the system in production were satisfied with the relevant procedures and documentation. Support acceptance: the people responsible for supporting the system once in production were satisfied with the relevant procedures and documentation. Cost and estimate acceptance: the current expenditures were acceptable, and reasonable estimates had been made for future production costs.

Project Development Issues

As the project progressed, the team ran into several issues, most of which are common in similar projects.

Personnel

The developers adopted the process pretty quickly because many of the object-oriented development practices were familiar to them already and they preferred an iterative and incremental development approach. However, the development team leader and the database administrator (DBA) had a “big design up front” (BDUF)—also known as “big modeling up front” (BMUF)¹¹—mindset that they started to overcome only after seeing that the emergent-design approach actually worked in practice. The business analyst tried hard to learn how to apply use cases effectively in combination with other requirements artifacts. One tester was

initially concerned that there weren't enough detailed models to inspect and review. However, he soon realized that the closer collaboration among the team and the combination of significant developer testing using the bank's testing tool and testing in the large activities such as system testing and UAT were sufficient.

Management

Senior management supported the project team throughout the effort because they could clearly see that the team was delivering. The project manager could therefore focus on issues such as scheduling and estimating and softer issues such as ensuring that the team had the resources they needed, including training, and that they weren't being delayed by politics.

Tools

The team could have been more agile regarding tool usage. It didn't fully accept the practice of "use the simplest tools": whiteboards made sense, but index cards and essential user interface prototypes using Post-it notes and flip chart paper were too foreign.

The Culture

The hardest thing to overcome was the serial/waterfall mindset of some participants. The development team leader and the tester from the responsible test center wanted more formal documentation. The DBA's data-driven mentality was typical but we eventually confronted it successfully, mostly because of the type of the project that focused on UI integration and legacy data presentation that necessarily had to go over multiple iterations. Senior management was already committed to the project because it had already participated in the project's definition and pushed for it.

Documentation

The team wrote too much documentation, but this increased its comfort level with the new approach, so it wasn't too bad. The team produced only the required AUP artifacts. The requirements model focused on use cases, business rules, sequence diagrams, UI specifications, technical requirements, and a glossary. The business analyst wrote and maintained these documents, keeping them up-to-date as the requirements evolved. The project manager maintained the software architecture document, which included formally transferring point form notes, whiteboard sketches, and arguably, models, in the computer-aided software engineering tool. The notes, basically an overview of

the team's decisions, and the whiteboard diagrams were quite agile. The testers carried out the testing with the Mercury testing and analysis tool. They inserted all the test cases (scenarios) and monitored testing progress using the tool. The testers also maintained the test model—the definition of the test cases—but invested too much effort in its development.

Lessons Learned

For a RUP project team to adopt AUP and AM techniques,^{10–11} it must overcome common developer misconceptions about RUP as well as several cultural barriers common in organizations that instantiate RUP. The team needed to start thinking outside of the current UML-centric nature of RUP.

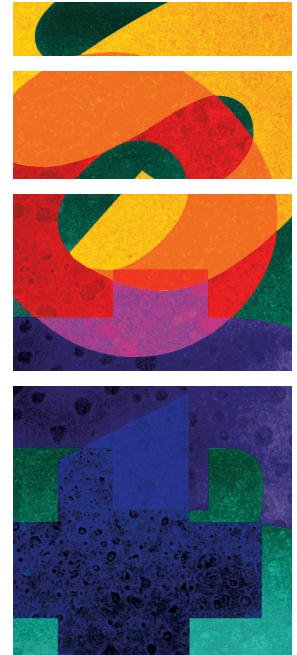
To do this, the team tried and mostly succeeded in the following eight goals.

First, they detached from the phrase "use-case driven." Use cases, elaborated by developers (playing also the business analyst role) weren't sufficient to drive the team to the final product. Use cases are good for documenting behavioral requirements, but that was only a part of the functional-requirements picture and a smaller part of the total requirements picture. Use cases aren't the best for documenting business rules, user interface requirements, constraints, or nonfunctional requirements. This is why RUP includes a supplementary specification to contain all these other things.

Second, they acknowledged that there are more modeling artifacts than those that UML offers. AM's multiple-models principle reminds developers that we have many modeling artifacts at our disposal, such as change cases, user stories, business rules, UML activity diagrams, UML class diagrams, data models, and external interface specifications. Many people perceive that RUP is simply a process for using UML. However, RUP recognizes that a wide range of models is needed to explore the complexities of modern software, and recent versions include data modeling and user interface design activities that are currently outside UML's scope.

Third, they recognized that RUP isn't inherently documentation-centric. RUP clarifies that only the required artifacts must be developed; however, many software professionals often neglect this. The team had to question every model that RUP suggests to determine the ones it needed for the project. It found AM's "travel light" and "model with a purpose" principles beneficial, as well as the "update only when it hurts" and "discard temporary models" practices.

Fourth, they created an agreed-upon process-



About the Authors



Ioannis T. Christou is an associate professor at Athens Information Technology and an adjunct professor at Carnegie-Mellon University's Information Networking Institute. His research interests include business intelligence and optimization algorithms and systems, enterprise software, and data mining. Christou has a PhD in computer sciences from the University of Wisconsin at Madison. He's a member of IEEE, the ACM, and the Technical Chamber of Greece. Contact him at ichr@ait.edu.gr.

Stavros T. Ponis is a lecturer in supply chain management at the National Technical University of Athens. He teaches a series of courses on supply chain management, operations research, e-commerce, and management of production information systems, enterprise-resource-planning systems, and operations research laboratories. Ponis has a PhD in Mechanical Engineering from the National Technical University of Athens. Contact him at staponis@central.ntua.gr.



Eleni Palaiologou is a senior project manager of major IT projects at one of the largest banks in Greece. Her research interests include information systems modeling, systems analysis, and process modeling and evaluation. Palaiologou has an MBA from the Athens University of Economics and Business in collaboration with the National Technical University of Athens. Contact her at elpalai@gmail.com.

based perspective for both developers and project stakeholders. Managers often tend toward prescriptive software processes such as RUP. Developers, on the other hand, tend toward agile techniques such as Extreme Programming and AM on the basis of the techniques' perceived focus on building software. Because management is the decision-making authority, many developers find themselves in a situation where management has chosen RUP and forces them to follow it. Fortunately, RUP is flexible and can be tailored to be reasonably agile. Still, developers and project stakeholders must agree on the extent of the tailoring.

Fifth, they successfully implemented the core disciplines of iterative and incremental development. Experienced modelers might have difficulty modeling AM practices such as “model in small increments,” “iterate to another artifact,” and “create several models in parallel.” Traditional modeling techniques often promote a single-artifact approach, such as use-case modeling or user-interface prototyping sessions, and a BDUF/BMUF approach in which you model everything in detail before coding. Although in theory, focusing on one artifact at a time should have enabled the modelers to get it right quickly, practice shows this isn't the case. Instead of use-case modeling sessions, the team tried to run requirements-modeling sessions in which they worked on use

cases, class responsibility collaborator cards, business rules, and user interface prototypes simultaneously. Similarly, they held analysis sessions where use-case modeling, sequence diagramming, UI prototyping, and class modeling made sense. They also held design sessions on class modeling, state modeling, data modeling, component modeling, UI prototyping, and even developing business code.

Sixth, they complied with the rule of simplicity. Simplicity is a fundamental value of AUP and AM in particular, promoting several critical principles that dramatically improved the effectiveness of our modeling efforts. Instead of specifying everything possible, we created sequence diagrams that were just good enough to depict the likely functionality of the automation of the daily tasks, and began coding from there. Agile modelers assume that programmers can figure out the details during development and therefore focus on issues that might not be so obvious. By keeping the models simple, the team worked faster while creating something of actual value to the programmers—models that focused on critical issues.

Seventh, they staffed the project with personnel who can contribute in both modeling and development. Many organizations have separate positions for modelers, motivating their staff to focus on specialties—a practice that in our experience reduces agility. Although RUP states clearly that individual developers can and should take multiple roles on a project, in general, most organizations adopting RUP don't follow this advice. They tend to introduce positions along the lines of the process's modeling roles—for example, requirements analyst, system analyst, UI designer, database designer—and therefore label people with unique roles, going against the advice of both RUP and AUP. People whose only job on a software project is to produce models tend to overmodel things for two reasons. First, they naturally want to do a good job. Second, there's a hand-off and therefore greater motivation to add more detail to the model, detail that likely wouldn't be needed if the people developing models also wrote the code.

Finally, they adapted AM's principle of honest, open communication. Development teams are usually reluctant to follow the “display models publicly” practice with people external to the team, often because they're afraid of what another political faction in the organization would do with that information. They initially hesitated to communicate the models to stakeholders outside the development team owing to anticipated criticism. Fortunately, this criticism never materialized. As soon

as the team displayed its models publicly, people realized there wasn't much to criticize, so the team greatly benefited from publicizing its models.

Overall, we found that to succeed in applying AUP, the organization's culture and management must be receptive to both RUP and to agile methods, and there lies a contradiction; that while most organizations adopting RUP target a fairly rigid and prescriptive process, organizations that adopt agile methods typically work in a much more flexible manner. Fortunately, we found that RUP is flexible enough that it can be customized to be reasonably agile. One immediate and direct implication this project had on the organization was the decision of the IT strategy department to include a customized version of AUP as the proposed methodology for all small-scale IT projects in the bank. 

References

1. P. Kroll and P. Kruchten, *The Rational Unified Process Made Easy: A Practitioner's Guide to the RUP*, Addison-Wesley, 2003.
2. R.D. Gibbs, *Project Management with the IBM Rational Unified Process: Lessons from the Trenches*, IBM Press, 2006.
3. S. Bergström and L. Råberg, *Adopting the Rational*

Unified Process: Success with the RUP, Addison-Wesley, 2004.

4. S. Ambler, "The Agile Unified Process (AUP)," Ambysoft, 2005; www.ambysoft.com/unifiedprocess/agileUP.html.
5. M. Sliger and S. Broderick, *The Software Project Manager's Bridge to Agility*, Addison-Wesley Professional, 2008.
6. J. Shore and S. Warden, *The Art of Agile Development*, O'Reilly Media, 2007.
7. M. Fowler and J. Highsmith, "The Agile Manifesto," *Software Development*, 2001; www.drdobbs.com/184414755.
8. C. Jefferies, P. Brereton, and M. Turner, "A Systematic Literature Review of Approaches to Reengineering for Multi-channel Access," *Proc. 12th European Conf. Software Maintenance and Reengineering*, IEEE CS Press, 2008, pp. 258–262.
9. S. Ambler, "An Introduction to Agile Modeling," Ambysoft, 2005; www.agilemodeling.com/essays/introductionToAM.htm.
10. S. Ambler, *Agile Modeling: Effective Practices for Extreme Programming and the Unified Process*, John Wiley & Sons, 2002.
11. S. Ambler, "Big Modeling Up-Front Anti-pattern," Ambysoft, 2005; www.agilemodeling.com/essays/bmuf.htm.
12. "Microsoft Customer Care Framework, the Fast Path to Reduced Costs and Increased Productivity," rev. 2.0, Accenture, 2008; http://origin.www.accenture.com/NR/rdonlyres/53058BEA-88B5-425C-8DE0-91DD70AFF2D3/0/Accenture_CCF.pdf



Selected CS articles and columns are also available for free at <http://ComputingNow.computer.org>.

ADVERTISER INFORMATION MAY/JUNE 2010 • IEEE SOFTWARE

Advertiser

Better Software 2010
Seapine Software Inc.

Page
Cover 3
Cover 4

Advertising Personnel

Marion Delaney
IEEE Media, Advertising Dir.
Phone: +1 415 863 4717
Email: md.ieeemedia@ieee.org

Marian Anderson
Sr. Advertising Coordinator
Phone: +1 714 821 8380
Fax: +1 714 821 4010
Email: manderson@computer.org

Sandy Brown
Sr. Business Development Mgr.
Phone: +1 714 821 8380
Fax: +1 714 821 4010
Email: sb.ieeemedia@ieee.org

Advertising Sales Representatives

Recruitment:

Mid Atlantic
Lisa Rinaldo
Phone: +1 732 772 0160
Fax: +1 732 772 0164
Email: lr.ieeemedia@ieee.org

New England
John Restchack
Phone: +1 212 419 7578
Fax: +1 212 419 7589
Email: j.restchack@ieee.org

Southeast
Thomas M. Flynn
Phone: +1 770 645 2944
Fax: +1 770 993 4423
Email: flynnntom@mindspring.com

Midwest/Southwest
Darcy Giovingo
Phone: +1 847 498 4520
Fax: +1 847 498 5911
Email: dg.ieeemedia@ieee.org

Northwest/Southern CA
Tim Matteson
Phone: +1 310 836 4064
Fax: +1 310 836 4067
Email: tm.ieeemedia@ieee.org

Japan
Tim Matteson
Phone: +1 310 836 4064
Fax: +1 310 836 4067
Email: tm.ieeemedia@ieee.org

Europe
Heleen Vodegel
Phone: +44 1875 825700
Fax: +44 1875 825701
Email: impress@impressmedia.com

Product:

US East
Dawn Becker
Phone: +1 732 772 0160
Fax: +1 732 772 0164
Email: db.ieeemedia@ieee.org

US Central
Darcy Giovingo
Phone: +1 847 498 4520
Fax: +1 847 498 5911
Email: dg.ieeemedia@ieee.org

US West
Lynne Stickrod
Phone: +1 415 931 9782
Fax: +1 415 931 9782
Email: ls.ieeemedia@ieee.org

Europe
Sven Anacker
Phone: +49 202 27169 11
Fax: +49 202 27169 20
Email: sanacker@intermediapartners.de